Graph Basics

Deep Dive + Traversals + Real-World Systems

Java 8+

Session Structure

1. What is a graph?
2. Graph terminology
3. Graph representations
4. BFS & DFS
5. Time & space complexity
6. Real-world applications
7. Interview-level problems

What is a Graph?

A graph is:

- A collection of vertices (nodes)
- Connected by edges

Used to model:

- Networks
- Relationships
- Dependencies

Graph Terminology

- Vertex (Node)
- Edge
- Directed vs Undirected
- Weighted vs Unweighted
- Path
- Cycle
- Connected components

Types of Graphs

1. Undirected Graph
2. Directed Graph
3. Weighted Graph
4. Cyclic Graph
5. Acyclic Graph (DAG)

Graph Representation

Two main ways:

1. Adjacency Matrix
2. Adjacency List

Adjacency Matrix

2D array:

```
int[][] matrix = new int[n][n];
```

Space: O(n²)

Pros:

- Fast edge lookup O(1)

Cons:

- High space usage

Adjacency List (Preferred)

```
java.util.List<java.util.List<Integer>> graph =
    new java.util.ArrayList<>();

for (int i = 0; i < n; i++)
    graph.add(new java.util.ArrayList<>());
```

Space: O(V + E)

Best for sparse graphs.

Add Edge (Undirected)

```
graph.get(u).add(v);
graph.get(v).add(u);
```

Breadth First Search (BFS)

Uses Queue.

Time: O(V + E)

Space: O(V)

```
static void bfs(int start,
        java.util.List<java.util.List<Integer>> graph) {

    boolean[] visited = new boolean[graph.size()];
    java.util.Queue<Integer> queue = new java.util.ArrayDeque<>();

    visited[start] = true;
    queue.offer(start);
```

```
    while (!queue.isEmpty()) {
        int node = queue.poll();
        System.out.print(node + " ");

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
}
```


Depth First Search (DFS)

Uses Recursion or Stack.

Time: O(V + E)

Space: O(V)

```
static void dfs(int node,
        java.util.List<java.util.List<Integer>> graph,
        boolean[] visited) {

    visited[node] = true;
    System.out.print(node + " ");

    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor])
            dfs(neighbor, graph, visited);
    }
}
```

BFS vs DFS

Feature	BFS	DFS
Structure	Queue	Stack/Recursion
Finds shortest path (unweighted)	Yes	No
Memory usage	Higher	Lower (usually)

Real World Use Case — Social Networks

Nodes = Users

Edges = Friend connections

Used to:

- Suggest friends
- Detect communities
- Find shortest connection

Real World Use Case — Navigation Systems

Road network:

Nodes = Locations
Edges = Roads

Shortest path algorithms:

- BFS (unweighted)
- Dijkstra (weighted)

Detect Cycle in Undirected Graph

Time: O(V + E)

```
static boolean hasCycle(int node, int parent,
        java.util.List<java.util.List<Integer>> graph,
        boolean[] visited) {

    visited[node] = true;

    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            if (hasCycle(neighbor, node, graph, visited))
                return true;
        } else if (neighbor != parent) {
            return true;
        }
    }
    return false;
}
```


Connected Components

Time: O(V + E)

```
static int countComponents(
        java.util.List<java.util.List<Integer>> graph) {

    boolean[] visited = new boolean[graph.size()];
    int count = 0;

    for (int i = 0; i < graph.size(); i++) {
        if (!visited[i]) {
            dfs(i, graph, visited);
            count++;
        }
    }
    return count;
}
```


Topological Sort (DAG)

Used for:

- Task scheduling
- Dependency resolution

Time: O(V + E)

Topological Sort (DFS Based)

```
static void topoDFS(int node,
        java.util.List<java.util.List<Integer>> graph,
        boolean[] visited,
        java.util.Stack<Integer> stack) {

    visited[node] = true;

    for (int neighbor : graph.get(node))
        if (!visited[neighbor])
            topoDFS(neighbor, graph, visited, stack);

    stack.push(node);
}
```


Graph Complexity Summary

Algorithm	Time
BFS	O(V + E)
DFS	O(V + E)
Cycle Detection	O(V + E)
Topological Sort	O(V + E)

Common Mistakes

- Not marking visited properly
- Infinite recursion
- Confusing tree vs graph
- Forgetting disconnected components
- Wrong graph representation choice

Interview Discussion Points

- Why O(V + E)?
- When to use adjacency matrix?
- BFS vs DFS difference?
- Detect cycle logic?
- Why visited array is needed?

Practice Problems

Easy:

- Number of islands
- Clone graph

Medium:

- Course schedule
- Detect cycle in directed graph

Advanced:

- Dijkstra’s algorithm
- Minimum spanning tree
- Word ladder

Summary

Graphs provide:

- Modeling of relationships
- Efficient traversal (O(V + E))
- Foundation of modern systems
- Core of networking & routing algorithms

Master:

- BFS
- DFS
- Cycle detection
- Connected components
- Topological sorting